

Cache High Availability

 systemdesignschool.io/fundamentals/cache-high-availability

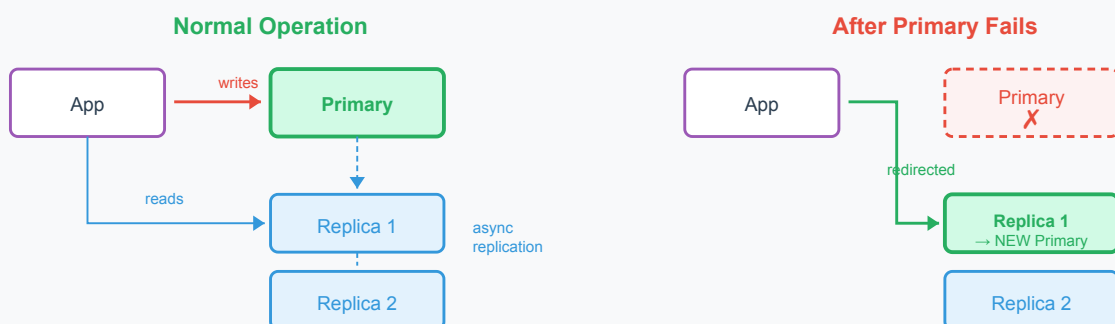


A sharded cache spreads keys across multiple nodes. But if one node crashes, all keys on that node become misses simultaneously. If that node held popular keys, the database absorbs a sudden spike. High availability keeps the cache serving even when individual nodes fail.

Replication: Primary-Replica

Each cache node (primary) has one or more replicas that mirror its data. The primary handles all writes and replicates them asynchronously to replicas. Reads can be served by either the primary or its replicas.

Primary-Replica: Failover on Crash



Sentinel detects failure, promotes replica in ~2-5 seconds (automatic)



When the primary fails, a replica contains a nearly-complete copy of the data. Promotion to primary is fast—the replica already holds the data in memory. The only data lost is writes that arrived at the primary after the last replication batch (typically milliseconds of data).

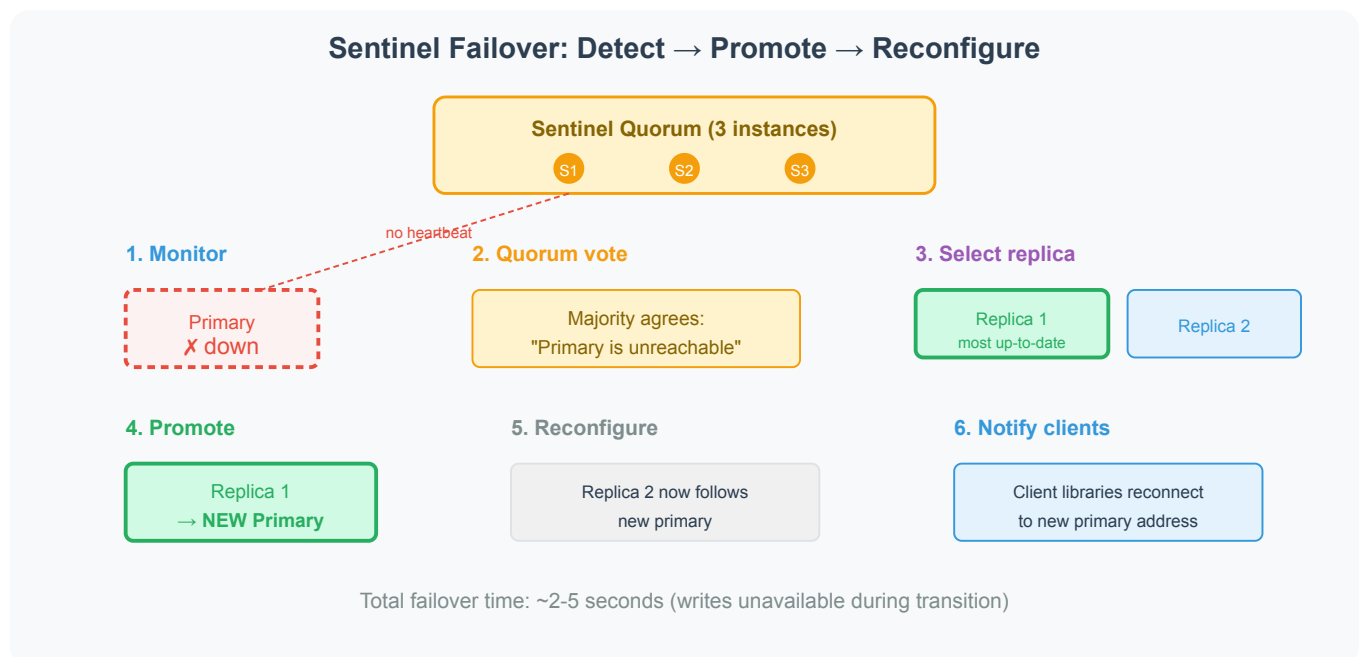
Read scaling. Replicas also serve as read capacity. A primary with 2 replicas handles roughly 3x the read throughput of a single node. The application reads from replicas for eventually-consistent data and from the primary when strong consistency is required.

Replication lag. Because replication is asynchronous, replicas can be slightly behind the primary. A write to the primary may not appear on the replica for milliseconds to seconds depending on load and network conditions. For cache use cases, this lag is typically acceptable—the cache already operates on eventual consistency assumptions.

Failover: Sentinel

When a primary fails, something must detect the failure and promote a replica. Redis Sentinel provides this automation.

Sentinel is a separate process (typically 3+ instances for quorum) that monitors primary and replica nodes. When a majority of Sentinels agree that a primary is unreachable, they trigger an automated failover.



The steps:

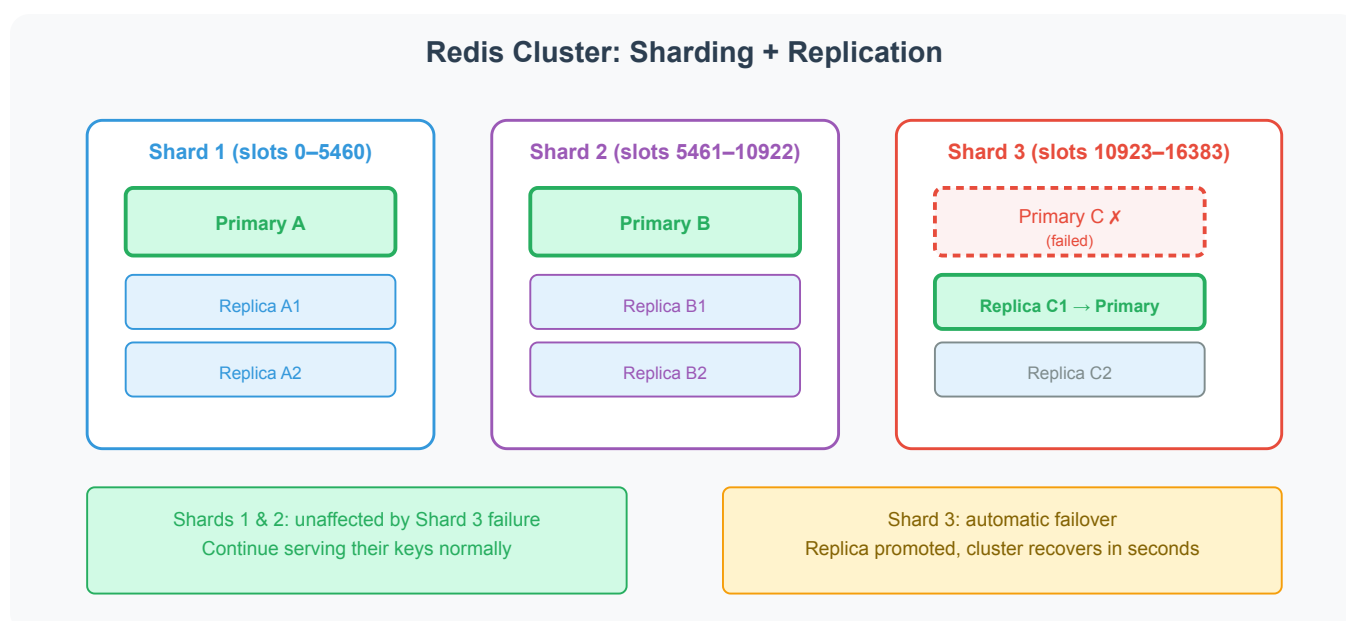
1. Select the replica with the most up-to-date data
2. Promote it to primary
3. Reconfigure the other replicas to follow the new primary
4. Notify client libraries of the topology change

The failover takes a few seconds—during which writes to the failed primary are lost and reads may fail. After promotion, the cluster resumes normal operation. Client libraries that support Sentinel reconnect automatically to the new primary.

Split-brain risk. If the old primary is not actually dead (network partition), both the old and new primary accept writes. When the partition heals, the old primary discovers it's been demoted and discards its writes. Data written to the old primary during the partition is lost. For cache workloads, this is acceptable—the data can be re-fetched from the database.

Redis Cluster: Built-In HA

Redis Cluster combines sharding and replication in one system. Each shard (set of hash slots) has a primary and one or more replicas. When a shard's primary fails, the cluster automatically promotes a replica—no separate Sentinel process needed.



The cluster remains available as long as each shard has at least one surviving node (primary or replica). If an entire shard (primary + all replicas) goes down, only the keys in that shard's slots become unavailable. Other shards continue serving normally.

Configuration	Nodes	Tolerates	Total Memory
3 primaries, 1 replica each	6	1 failure per shard	3x shard size
3 primaries, 2 replicas each	9	2 failures per shard	3x shard size
6 primaries, 1 replica each	12	1 failure per shard	6x shard size

The `cluster-require-full-coverage` configuration controls behavior when a shard is completely down. Set to `yes` (default), the cluster stops serving key operations (both reads and writes) and reports `CLUSTERDOWN`. Set to `no`, the surviving shards continue serving their keys while the failed shard's keys return errors.

Cache as a Non-Critical Layer

Caches differ from databases in one important way: the database is the source of truth. If the cache fails entirely, the application can fall back to direct database queries. It will be slower, but no data is lost.

This changes the HA calculus. For a database, losing a node risks data loss—stronger durability guarantees (sometimes synchronous replication) may be required depending on RPO. For a cache, losing a node risks only performance degradation. Asynchronous replication (faster, simpler) is acceptable because the worst case is re-fetching from the database.

Graceful degradation. When a cache node fails, the application should handle the resulting misses without cascading failures. Circuit breakers prevent the database from being overwhelmed by the sudden miss storm. Rate limiting on cache-miss paths ensures the database handles the spike without crashing. The cache re-warms naturally as requests flow through.

From here, the cold start problem appears when a cache node starts empty, and warming strategies prevent the database from getting overwhelmed.